INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

Assignment 04: Project FlightOps

Tool Calling & Building AI Agents

Course: Agentic AI: From Concepts to Practice
Instructor: Prof. Karthik Vaidhyanathan
TAs: Aviral Gupta, Aneetta Sara Shany, Ch Pavan Harshit, Shreyash Chandak

Topic: Tool Calling & Building AI Agents
Deadline: 23rd August 2026, 11:59 PM

Mission Briefing

TO: Lead AI Engineer
FROM: Director of Flight Operations, AeroWing Airlines
SUBJECT: AeroBrain Knows Everything. It Does Nothing.

AeroBrain has been live for two weeks and the grounding problem is solved. Pilots ask about ramp weights, seat pitches, and cargo dimensions, and they get accurate, cited answers straight from the manual. Operations is impressed. Unfortunately, they now expect more, and AeroBrain cannot deliver it.

Yesterday someone asked, *"Is Flight AI203 delayed?"* AeroBrain replied, *"Please check with Operations."* Someone else asked, *"Find me an empty gate."* It replied, *"I cannot perform this action."* A third person asked it to generate today's maintenance report, and it politely refused. In every case, the answer to the question exists somewhere in our systems – AeroBrain simply has no way to reach it.

The Diagnosis: AeroBrain can only read. It cannot act. A retrieval system can tell a pilot what the manual says about the A320; it cannot tell them whether *today's* A320 at gate B12 is airworthy, because that answer does not live in a PDF – it lives in our flight, maintenance, and gate databases, changing by the minute.

Your Mandate: Upgrade AeroBrain into **FlightOps**: an AI Operations Officer that can call our internal software directly – checking flight status, finding an open gate, pulling a maintenance history – and reason about which of these to use, and in what order, before it ever opens its mouth.

Learning Objectives

By the end of this assignment you should be able to

- Explain what tool calling is, and why an LLM cannot execute your Python functions on its own.

- Write a function declaration (name, description, parameters, required fields) and explain why the model's tool choice depends entirely on how well it is worded.
- Implement the full function-calling lifecycle: request, execute, respond, and receive a final answer.
- Build an agent loop that chains multiple tool calls together to answer one complex question.
- Make an agent generate a goal and a plan before it acts, and reflect on its own tool usage after it answers.

Assignment Tasks

A Note Before You Start: No Notebooks

Previous assignments in this course have been submitted as single Google Colab notebooks. This assignment is different. A tool-calling agent is, at its core, a small software system composed of multiple components that interact with and call one another. Structuring the agent across separate Python files makes the system easier to understand, debug, test, and reuse.

This assignment must not be submitted as a Google Colab notebook.

Instead, you are expected to submit a ZIP file containing appropriately structured and scoped Python files that together implement the complete functionality of the agent. Your submission must include:

- **README.md** – documenting the overall approach and describing the functionality and responsibility of each file.
- **main.py** – serving as the entry point for your final artifact and orchestrating the overall execution of the agent.

Additional Python files may be included as needed to keep the implementation modular and well organized.

Model/provider choice: You may use any language model API that supports tool/function calling. Your `README.md` must state which model provider and model you used. Provider-specific code should be isolated in a small, clearly identified portion of the project so that the rest of the agent architecture remains model-agnostic.

Part 1: Building the Airline Backend

Objective: Give FlightOps something real to act on.

Before FlightOps can call a tool, that tool has to exist. In `data.py`, create mock dictionaries for *Flights*, *Aircraft*, *Passengers*, *Maintenance*, and *Weather* – populate each with a handful of realistic entries. In `tools.py`, implement **5–7 plain Python functions** that read from these dictionaries. Suggested tools:

1. `get_flight_status(flight_number)` – returns status, gate, departure time, and delay.
2. `search_passenger(name)` – returns booking reference, seat, and destination.
3. `maintenance_history(tail_number)` – returns last inspection date, hours flown, and outstanding issues.

4. `find_available_gate(terminal)` – returns an open gate number.
5. `get_weather(airport)` – returns visibility, wind, and temperature.
6. `lookup_aircraft(type)` – returns dimensions, capacity, and fuel figures.

Test every function directly from `app.py` before you connect a single one to the model. If a tool is broken, you want to know that now – not while debugging a tangled conversation with the model.

Part 2: Declaring Your Tools

Objective: Describe each tool well enough that the model can choose correctly between them.

For every function in `tools.py`, write a matching declaration in `schemas.py`: its **name**, a **description** of what it does and when to use it, its **parameters**, and which of those are **required**.

The description is doing more work than it looks like. The model does not directly inspect your Python implementation – it relies on the tool declaration/schema you provide. Two tools with vague, overlapping descriptions ("gets flight info" and "gets flight data") will get confused for each other; a precise description ("returns the current status, gate, and delay for a specific flight number, e.g. AI203") will not. Write two versions of one tool's description – a lazy one and a careful one – and note in your README which produced better tool selection and why.

Part 3: Implementing the Function Calling Loop

Objective: Connect the model's decisions to your code's execution.

This is the heart of the assignment. In `agent.py`, build the lifecycle that turns a user's message into a grounded, acted-upon answer:

1. Send the user's message to a language model, along with your tool declarations.
2. The model determines whether a tool is needed and, if so, requests a specific tool with the required arguments.
3. Your code parses the tool name and its arguments.
4. Your code finds the matching function in `tools.py` and executes it.
5. Your code captures the result and provides it back to the model as a tool result.
6. The model uses that result to produce the final, natural-language answer.

The Function Calling Loop

Ask → Tool request → Run tool → Return result → Model answers

The model never executes your Python functions. Your program executes them. That is the entire point of function calling – the developer, not the LLM, controls every side effect.

Critical Incident Report

FlightOps is live for its first real test. Asked *"Is AI203 likely to depart on time?"*, it calls `get_flight_status("AI203")`, reads back the scheduled departure time, and confidently tells the operations desk the flight will leave on schedule.
Ten minutes later the flight is delayed two hours – fog rolled in at the departure airport, and there is an open maintenance item on the aircraft that was about to be assigned to that gate. FlightOps never checked either.
One tool call is not one answer. A single function told FlightOps the flight's *scheduled* status, and it stopped there – exactly like a retriever that returns the top-ranked chunk and never questions whether it is the whole story. Answering a real operational question means combining several tools and reasoning about what they collectively imply.

Part 4: The Multi-Tool Agent Loop

Objective: Chain several tool calls together to answer questions no single tool can answer alone.

Some questions – like the one that tripped up FlightOps above – genuinely require more than one tool. To answer *"Is AI203 likely to depart on time?"* properly, your agent should reason through something like `get_weather()` → `get_flight_status()` → `maintenance_history()` → a synthesized answer.

Implement this as a loop in `agent.py`, rather than a single request/response pair:

```
while the model requests a tool:
    execute the requested tool
    append the tool response to the conversation
    send the updated conversation back to the model
return the final text answer
```

The implementation may use any model or API that supports tool/function calling. Keep provider-specific integration code isolated from the rest of your application so that the agent architecture is not tightly coupled to one model provider.

Test it on at least one question that genuinely needs three or more tool calls, and log the sequence of calls your agent actually made.

Exploration (Not Graded)

Does giving the model access to *more* tools than a question needs ever hurt it – does it call tools it did not need to, or take longer to converge on an answer? Try running the same question against a version of your agent with only three tools available, and against the full six. What changes?

Part 5: Planning

Objective: Make the agent think before it acts, and show its thinking.

Before any tool is called, use `planner.py` to generate a **Goal** and a **Plan** for the user's question, and display that plan before the agent starts executing tools. For example:

```
Goal:
Determine if AI203 can depart on time.

Plan:
1. Check weather at the departure airport.
2. Check the aircraft's maintenance status.
3. Check gate availability.
4. Produce a summary for the user.
```

The plan does not need to be followed rigidly step-by-step by the agent loop – its purpose is transparency. An operations officer reading FlightOps' output should be able to see *why* it checked what it checked.

Part 6: Reflection

Objective: Have the agent evaluate its own performance after it answers.

After the final answer is produced, run a separate prompt through `reflector.py` that asks the model to critique its own process. Suggested reflection questions:

- Were any tools called that turned out to be unnecessary?
- Was any information the user needed still missing from the final answer?
- Could the same answer have been reached with fewer tool calls?
- How confident should the user be in this answer, and why?

Display the reflection alongside the final answer – do not hide it. The point is for the student (and the user) to see the agent reasoning about its own limitations, not just its conclusions.

Exploration (Not Graded)

Bonus: build a simple trace visualization of one full run – Goal → Plan → Tool → Observation → Tool → Observation → Answer → Reflection – either printed to the console in a readable format or rendered in a minimal web page.

Part 7: Error Handling

Objective: Make sure one broken tool does not take down the whole agent.

At least one tool must be able to fail – simulate an unavailable weather service, a malformed tail number, whatever fits. Catch the exception inside the tool itself and return a structured error instead of letting the program crash:

```
{
    "status": "error",
    "message": "Weather service unavailable"
}
```

That error result still gets sent back to the model like any other tool result. Confirm that the model explains the failure to the user in plain language, rather than the agent loop throwing an unhandled exception.

CONSTRAINTS: BUILD IT YOURSELF

No **LangChain**. No **CrewAI**. No **AutoGen**. No **MCP**. No agent frameworks of any kind.

Everything – the tool loop, the planner, the reflector – must be implemented by hand, with plain function calls and plain Python control flow. You are meant to understand exactly what an agent framework is automating for you, before you ever let one automate it.

Final Report

Answer the following in your `README.md`. Each answer should be three to five sentences.

1. In Assignment 03, grounding meant retrieving the right passage of a document. Here, grounding means calling the right function. What is genuinely different about the risk profile of the second kind of grounding, compared to the first?
2. A function and its declaration can silently drift apart – you rename a parameter in `tools.py` but forget to update `schemas.py`. What would actually happen when the model tries to call that tool, and how would you detect the mismatch before a user does?
3. Suppose AeroWing later wants FlightOps to *cancel* a passenger's booking or *reassign* a gate, not just look information up. What changes about the risk of letting the agent call that tool autonomously, and what safeguard would you add before shipping it?
4. Which mattered more for getting the right tool called at the right time in your experiments: the quality of your function descriptions, or the quality of your system prompt? Support your answer with a specific observation from Part 2 or Part 4.

Submission Guidelines

- Submit a single zip file consisting of
  - your complete Python project (as described above),
  - your `README.md`, including architecture notes, run instructions, example prompts, and your Final Report answers,
  - two screenshots in `screenshots/`: one showing a multi-tool run with its plan and reflection visible, and one showing the agent handling the failing tool gracefully. (Titled `partX.jpeg` or `partX.png`)
- Ensure your project runs from the command line, end to end, without errors on a clean

checkout.

- Configure your chosen model provider through environment variables, loaded via `config.py`. Do **not** hardcode it, and do **not** include your `.env` file in the submission – include a `.env.example` instead.
- Submit the file using the filename:
  `FlightOps_YourName.zip`

An AI that only knows things is a library. An AI that can act on what it knows is a colleague. Give AeroBrain hands, and it becomes FlightOps.